2

Setting Up WhatsPackt's Messaging Abilities

In the previous chapter, we created the structure and UI needed for our messaging app, WhatsPackt.

One of the core features of any messaging app is the ability to facilitate 1:1 conversations between two users, so in this chapter, we will delve into the process of connecting our messaging app to a backend server using WebSockets, handling messages within `ViewModel` instances, and managing synchronization, error handling, and push notifications.

We will begin by exploring **WebSockets**, a powerful technology that enables bidirectional communication between client and server, providing a solid foundation for real-time messaging in your app. You will learn how to establish a WebSocket connection, send messages, and handle incoming messages from the server.

Next, we will demonstrate how to receive messages in your **ViewModel**. We will discuss best practices for updating the UI, managing message storage, and handling user interactions, ensuring a smooth and responsive messaging experience for your users.

The chapter will also cover the essential aspects of synchronization and error handling. You will learn how to manage message delivery status, handle intermittent connectivity issues, and gracefully recover from errors, resulting in a resilient and reliable messaging system.

Finally, we will dig into the topic of push notifications, which are vital for alerting users of new messages even when the app is not in the foreground.

By the end of this chapter, you will have gained a comprehensive understanding of the key components and technologies involved in creating a modern messaging app that supports 1:1 conversations using WebSockets and push notifications.

So, in this chapter, we will cover the following topics:

- Using a WebSocket connection
- Receiving messages in our `ViewModel`
- Handling synchronization and errors
- Adding push notifications
- Replacing the WebSocket with Firestore

Technical requirements

As in the previous chapter, you will need to have installed Android Studio (or another editor of your preference).

We are also going to assume that you followed along with the previous chapter. If you haven't, you can download the previous chapter's complete code from here: **https://github.com/PacktPublishing/Thriving-in-Android-Development-using-Kotlin/tree/main/Chapter-1/WhatsPackt**.

The code completed in this chapter can also be found at this link: **https://github.com/PacktPublishing/Thriving-in-Android-Development-using-Kotlin/tree/main/Chapter-2/WhatsPackt**.

Using a WebSocket connection

As mentioned, WebSockets is a powerful technology that enables bidirectional communication between client and server. In this section, we are going to use a WebSocket connection to connect with our server to obtain and send messages. But before we do that, it is essential to understand the alternatives and the rationale behind choosing WebSockets for our messaging app.

Why WebSockets?

There are several options for enabling real-time communication between clients and servers, including the following:

- **Long polling**: This is when the client sends a request to the server, and the server holds the request until new data is available. Once the server responds with the new data, the client sends another request, and the process repeats.
- **Server-Sent Events (SSE)**: SSE is a unidirectional communication method where the server pushes updates to the client over a single HTTP connection.
- **Real-time cloud databases** (for example, Firebase Firestore): Real-time cloud databases provide an easy-to-use, scalable solution for real-time data synchronization. They automatically push updates to clients whenever data changes, making them suitable for messaging apps.
- **WebSockets**: WebSockets provide full-duplex, bidirectional communication between clients and servers over a single, long-lived connection. They are widely supported across platforms and are an ideal choice for real-time communication in messaging apps.

Considering these alternatives, we choose to use WebSockets for our messaging app because they offer the following advantages:

- **Bidirectional communication**: WebSockets enable simultaneous data transmission between clients and servers, allowing for faster message exchanges and a more responsive user experience

- **Low latency**: Unlike long polling, SSE, and some real-time cloud databases, WebSockets provide low-latency communication, which is crucial for a real-time messaging app
- **Efficient use of resources**: WebSockets maintain a single connection per client, reducing the overhead on both client and server compared to long polling
- **Flexibility and control**: Implementing custom WebSocket communication allows for more fine-grained control over the messaging infrastructure, avoiding potential limitations or constraints imposed by real-time cloud databases

For sure, WebSockets also have their disadvantages that we must take into account, such as the following:

- **Battery and data usage**: Maintaining a persistent connection can lead to increased battery drain and data usage, which may be a concern for mobile users.
- **Complexity**: Implementing WebSocket communication is typically more complex than using RESTful services. You have to handle various scenarios such as reconnection on network changes, which are common in mobile environments.
- **Scalability**: If your application scales to a large number of users, maintaining WebSocket connections for all of them can be resource-intensive on the server side.

Although there are some disadvantages, the advantages of using WebSockets — such as real-time bidirectional communication and lower overheads compared to traditional HTTP polling — significantly outweigh these issues, making them a powerful choice for interactive applications.

Let's start learning how we can integrate WebSockets.

Integrating WebSockets

There are several libraries available for integrating WebSockets in Android applications. Some popular options include the following:

- **OkHttp**: A popular HTTP client for Android and Java applications that also supports WebSocket communication
- **Scarlet**: A declarative WebSocket library for Kotlin and Java applications, built on top of OkHttp
- **Ktor**: A modern, Kotlin-based framework for building asynchronous servers and clients, including WebSocket support

For our app, we will use Ktor due to its ease of use, native support for Kotlin, and extensive documentation.

What is Ktor?

Ktor stands out for its coroutine-based architecture, which allows for non-blocking asynchronous operations, making it particularly suitable for I/O-intensive tasks such as network communication. It's lightweight and modular, allowing developers to pick and choose only the features they need, thereby avoiding the overhead of unnecessary functionality.

The framework is built on top of coroutines, a feature in Kotlin that makes your code cleaner and more readable, and simplifies asynchronous programming by allowing functions to be paused and resumed at later times. This provides a powerful way to handle concurrency with a more straightforward and expressive syntax compared to traditional callback mechanisms.

Ktor is versatile, supporting both server-side and client-side development. On the server side, it can be used to build robust and scalable web applications and services. On the client side, it provides a multiplatform HTTP client that can be used on Android, allowing for seamless interaction with web services.

Ktor's WebSocket client allows for easy setup and management of WebSocket connections, handling complexities such as connection life-

cycle, error handling, and message processing. Its **domain-specific language** (**DSL**) provides a concise and expressive way to define the behavior of WebSocket interactions, making the code more readable and maintainable.

Integrating WebSockets with Ktor

To integrate Ktor in our Android app, follow these steps:

1. In our app's **`build.gradle`** file of the **`:feature:chat`** module, add the following Ktor dependencies for the WebSocket client. Make sure to replace **`$ktor_version`** with the latest version available (for the examples in this book, we are using version 2.2.4):

```
dependencies {
    implementation "io.ktor:ktor-client-
        websockets:2.2.4"
    implementation "io.ktor:ktor-client-okhttp:2.2.4"
}
```

   Each dependency serves a distinct purpose:

   1. **`io.ktor:ktor-client-websockets`**: This dependency provides the necessary functionality to manage WebSocket connections in our application. It includes high-level abstractions for opening, sending messages to, and receiving messages from WebSocket servers, facilitating real-time data exchange in a seamless manner. By using this library, we can easily implement WebSocket communication without handling the complex underlying protocols and handshakes manually.
   2. **`io.ktor:ktor-client-okhttp`**: While Ktor is a multiplatform framework, it requires an engine to handle network requests. This dependency integrates OkHttp as the underlying engine for handling HTTP requests and responses in Android applications. OkHttp supports WebSockets along with its robust HTTP client features, providing efficient network operations, connection pool-

ing, and a powerful interface for making and intercepting requests.

Together, these dependencies allow our app to utilize WebSockets for real-time communication and leverage OkHttp's efficient networking capabilities. This combination is particularly powerful for applications needing to maintain persistent connections and manage high-frequency data exchange, such as messaging apps or live data feeds.

2. In your **`AndroidManifest.xml`** file, add the required permission to access the internet as we will need it to connect our WebSocket and receive/send messages:

```
<uses-permission android:name=
    "android.permission.INTERNET" />
```

We now have the library included in our project. As we will be using Ktor with Kotlin Flow, let's introduce it before diving into our WebSocket implementation.

Getting to know Kotlin Flow

Flow is part of Kotlin's coroutines library, and it's a type that can emit multiple values sequentially, as opposed to suspend functions that return only a single value. Flow builds upon the foundational concepts of coroutines to offer a declarative way to work with asynchronous streams of data.

Unlike sequences in Kotlin, which are synchronous and blocking, Flow is asynchronous and non-blocking. This makes Flow ideal for handling a continuous stream of data that can be observed and collected asynchronously, such as real-time messages from a WebSocket.

When integrating Flow with Ktor WebSockets, we can create a powerful combination where the WebSocket messages are emitted as a stream of data that can be processed using all the Flow operators. It allows for a

clean, reactive-style approach to handling incoming and outgoing messages with WebSockets.

For example, in a chat application, incoming messages from a WebSocket can be represented as a flow of strings. The app can collect this flow to update the UI accordingly. Similarly, user actions that generate outgoing messages can be collected and sent through the WebSocket connection.

The Flow API is really simple and easy to use. As another example, imagine that we have a flow that emits three strings:

```
fun main() = runBlocking {
    // Define a simple flow that emits three strings
    val helloFlow = flow {
        emit("Hello")
        emit("from")
        emit("Flow!")
    }
    // Collect and print each value emitted by the flow
    helloFlow.collect { value ->
        println(value)
    }
}
```

In this code block, **`helloFlow`** is defined, using the **`flow`** builder to emit three strings one after another.

NOTE

There are several other builders apart from **`flow`**, *such as* **`flowOf`**, *which creates a flow from a set of values, or* **`toFlow()`**, *which creates a flow from a collection.*

The **`collect()`** function is then called on **`helloFlow`**. It acts as a subscriber that reacts to each emitted value by printing it.

If you run this code, you should see the following output:

```
Hello
from
Flow!
```

Now that we are a bit familiar with Kotlin Flow, we are ready to do the next step: build our implementation of a WebSocket using Ktor and Flow. As it is going to be one of the data sources that will provide messages to our app, we will call it **`WebsocketDataSource`**.

Implementing WebSocketDataSource

To implement the WebSocket data source, we are first going to create an **`HttpClient`** instance. **`HttpClient`** is a Ktor class that allows you to make HTTP requests and manage network connections. In the case of WebSockets, it is responsible for establishing and maintaining the connection between the client and server.

To create an **`HttpClient`** instance with WebSocket support, we are going to create a new file called **`WebSocketClient`** in the **`feature.chat.data.network`** package (you will need to create the data and network packages as they don't exist yet) and include the following code:

```
object WebsocketClient {
    val client = HttpClient(OkHttp) {
        install(WebSockets)
    }
}
```

Here, we're using the **`OkHttp`** engine to create an **`HttpClient`** instance, and then we're installing the **`WebSockets`** plugin to enable WebSocket support.

NOTE

In Ktor, **plugins** *(also called features) are modular components that extend the functionality of Ktor applications. Plugins can be installed on both the client and server sides to provide additional features, such as authentication, logging, serialization, or custom behavior. Ktor's plugin-based architecture encourages a lightweight and modular approach, allowing you to include only the necessary components in your application.*

Then, we will create our **`MessagesSocketDataSource`** class (in the same package).

To start creating our WebSocket, we will need a **`WebSocketSession`** instance. **`WebSocketSession`** represents a single WebSocket connection between the client and server, providing methods for sending and receiving messages, as well as managing the connection's lifecycle. In our implementation, we will create a **`WebSocketSession`** instance when we call the **`connect()`** method, like so:

```
class MessagesSocketDataSource @Inject constructor(
    private val httpClient: HttpClient,
) {
    private lateinit var webSocketSession:
        DefaultClientWebSocketSession
    suspend fun connect(url: String): Flow<Message>{
        return httpClient.webSocketSession { url(url) }
            .apply { webSocketSession = this }
            .incoming
            .receiveAsFlow()
            .map{ frame ->
                webSocketSession.handleMessage(frame) }
            .filterNotNull()
            .map { it.toDomain() }
    }
//...
}
```

Let's break down what this code is going to do:

- **`suspend fun connect(url: String): Flow<Message>`**: The **`connect`** function is defined as a suspending (**`suspend`**) function that takes a **`url`** parameter of type **`String`** and returns a **`Flow<Message>`** instance. **`Flow`** is a cold asynchronous stream used for processing data in a reactive way in Kotlin (a cold stream is one that will only emit messages when there is a consumer connected).
- **`httpClient.webSocketSession { url(url) }`**: This line uses **`httpClient`** to create a WebSocket session by calling the **`webSocketSession`** function and passing a lambda that sets the session's URL to the provided URL.
- **`.apply { webSocketSession = this }`**: This line stores the newly created WebSocket session using the **`apply`** function in the **`webSocketSession`** property. We also need to store it as we will need the session later for sending messages.
- **`.incoming`**: This line accesses the incoming property of **`webSocketSession`**. The incoming property is a channel that receives incoming **`Frame`** objects from the WebSocket server.
- **`.receiveAsFlow()`**: This line converts the incoming channel to a **`Flow<Frame>`** instance so that it can be processed using the Flow API.
- **`.map { frame -> webSocketSession.handleMessage(frame) }`**: This line maps each incoming **`Frame`** object to the result of calling the **`handleMessage`** function. We will define the **`handleMessage`** function later.
- **`.filterNotNull()`**: This line filters out any **`null`** values from the stream, ensuring that only non-**`null`** values are processed further.
- **`.map { it.toDomain() }`**: This line maps each non-**`null`** value to the result of calling the **`toDomain()`** function. This function will map the current data-related object to the domain **`Message`** model that we will create soon.

Before processing and handling the messages, we will also want to add two more functions to our WebSocket data source:

- We want one function to send messages, as we want our users to be able to send messages to their WhatsPackt friends
- We want another function to disconnect the WebSocket, as we should disconnect it from the server when it is not in use

We can add these like so:

```
suspend fun sendMessage(message: String) {
    webSocketSession.send(Frame.Text(message))
}
suspend fun disconnect() {
    webSocketSession.close(CloseReason(
        CloseReason.Codes.NORMAL, "Disconnect"))
}
```

When a WebSocket connection is closed, it's accompanied by a **`CloseReason`** class, which contains a code and an optional descriptive text. The code indicates the reason for the connection closure, such as normal closure, protocol error, or unsupported data. In our implementation, we use the **`CloseReason`** class to close the **`WebSocketSession`** with a normal closure.

Some common **`CloseReason`** codes include the following:

- **`CloseReason.Codes.NORMAL`**: Indicates a normal closure of the connection. This is the reason that will be provided when the user is no longer using the chat screen.
- **`CloseReason.Codes.GOING_AWAY`**: Indicates that the server is going away or shutting down.
- **`CloseReason.Codes.PROTOCOL_ERROR`**: Indicates that an error in the WebSocket protocol occurred.
- **`CloseReason.Codes.UNSUPPORTED_DATA`**: Indicates that the received data type is not supported.

Now that we know how to close our WebSocket connection, we need to define the **`handleMessages`** extension function to process all the mes-

sages while the connection is alive:

```
private suspend fun
DefaultClientWebSocketSession.handleMessage(frame: Frame):
WebsocketMessageModel? {
    return when (frame) {
        is Frame.Text -> converter?.deserialize(frame)
        is Frame.Close -> {
            disconnect()
            null
        }
        else -> null
    }
}
```

In the WebSocket protocol, data is transmitted in discrete units called frames. Ktor provides a **`Frame`** class to represent these units, with different subclasses for each frame type, such as **`Frame.Text`**, **`Frame.Binary`**, **`Frame.Ping`**, and **`Frame.Close`**.

In our case, we are only processing **`Frame.Text`** and **`Frame.Close`** messages. To receive a **`Frame.Close`** message, we will close the WebSocket (for now – in the future, maybe we would want to do a retry here or give feedback about the problem to the user). Then, to receive the **`Frame.Text`** messages, we are going to **deserialize** them from JSON (a lightweight data-interchange format that is commonly used for communication between systems) to the object we are going to work with. Here, **`deserialize`** just describes this conversion.

We can configure a converter in our WebSocket that allows us to easily deserialize our messages. First, we need to add new dependencies to our **`build.gradle`** file:

```
implementation("io.ktor:ktor-serialization-kotlinx-json:2.2.4)
```

Then, we are ready to set **`contentConverter`** in our WebSocket plugin:

```
object WebsocketClient {
    val client = HttpClient(OkHttp) {
        install(WebSockets) {
            contentConverter =
               KotlinxWebsocketSerializationConverter(Json)
        }
    }
}
```

In this case, we are configuring the **`kotlinx.serialization`** converter for the JSON format (there are also converters available for other standards, such as XML, Protobuf, and CBOR).

In addition, we must add the **`@Serializable`** annotation to those data classes that we want to be deserialized by the converter. In our case, we will create a **`WebsocketMessageModel`** class as follows:

```
@Serializable
class WebsocketMessageModel(
    val id: String,
    val message: String,
    val senderName: String,
    val senderAvatar: String,
    val timestamp: String,
    val isMine: Boolean,
    val messageType: String,
    val messageDescription: String
)
```

The last step in our flow chain is to convert the **`WebsocketMessageModel`** class to a domain. As we still don't have a domain model, we should create it first:

```
data class Message(
```

```
    val id: String,
    val senderName: String,
    val senderAvatar: String,
    val timestamp: String,
    val isMine: Boolean,
    val contentType: ContentType,
    val content: String,
    val contentDescription: String
) {
    enum class ContentType {
        TEXT, IMAGE
    }
}
```

Now, we can implement the mapper as a function of the **`WebsocketMessageModel`** class:

```
@Serializable
class WebsocketMessageModel(
    val id: String,
    val message: String,
    val senderName: String,
    val senderAvatar: String,
    val timestamp: String,
    val isMine: Boolean,
    val messageType: String,
    val messageDescription: String
) {
    companion object {
        const val TYPE_TEXT = "TEXT"
        const val TYPE_IMAGE = "IMAGE"
    }
    fun toDomain(): Message {
        return Message(
            id = id,
            content = message,
            senderAvatar = senderAvatar,
            senderName = senderName,
            timestamp = timestamp,
            isMine = isMine,
```

```
            contentDescription = messageDescription,
            contentType = toContentType()
        )
    }
    fun toContentType(): Message.ContentType {
        return when(messageType) {
            TYPE_IMAGE -> Message.ContentType.IMAGE
            else -> Message.ContentType.TEXT
        }
    }
}
```

Here, we are adding the **`toDomain()`** function that maps the current **`WebsocketMessageModel`** class to the **`Message`** model. Note that almost all fields in the data model are similar to those in our domain **`Message`** model. The key exception is the **`messageType`** field, which we must convert to the enum we are using in the domain **`Message`** model. To simplify this conversion, we use the **`toContentType()`** function, which specifically transforms **`messageType`** from a **`String`** object to a **`ContentType`** enum.

We also would need to convert the domain **`Message`** object to the **`WebsocketMessageModel`** class. To do that, we need to add a new function to the **`WebsocketMessageModel`** class:

```
companion object {
    const val TYPE_TEXT = "TEXT"
    const val TYPE_IMAGE = "IMAGE"
    fun fromDomain(message: Message): WebsocketMessageModel {
        return WebsocketMessageModel(
            id = message.id,
            message = message.content,
            senderAvatar = message.senderAvatar,
            senderName = message.senderName,
            timestamp = message.timestamp,
            isMine = message.isMine,
            messageType = message.fromContentType(),
            messageDescription = message.contentDescription
```

```
            )
        }
    }
```

Here, we are converting the **`Message`** domain object into a **`WebsocketMessageModel`** class.

Then, in the **`send`** function, we will proceed as follows:

```
suspend fun sendMessage(message: Message) {
    val websocketMessage =
        WebsocketMessageModel.fromDomain(message)
    webSocketSession.converter?
        .serialize(websocketMessage)?.let
    {
        webSocketSession.send(it)
    }
}
```

With these changes to the **`sendMessage`** function, we are now receiving a domain model object, converting it to **`WebsocketMessageModel`**, and finally serializing it into a **`Frame`** object and sending it through our WebSocket.

The next step is to connect this component (**`MessagesWebsocketDataSource`**) with **`ViewModel`**, which will be responsible for providing the view state to the view so that it can render accordingly.

Receiving messages in our ViewModel

Our app is ready to receive and send messages using a WebSocket. Now, we need to make them reach the UI we created in the previous chapter. We will do that in this section, but first, we need to think about the architecture and components needed to do that.

Understanding Clean Architecture implementation

In the previous chapter, we modularized our app and talked about using a Clean Architecture-based structure to organize our common and feature modules. We have already created our first component of this architecture, `MessagesWebsocketDataSource`, but it is important to understand the reasons behind this organization and which role every component plays in the architecture.

There are extensive books, articles, and videos about why and how to apply Clean Architecture principles to an Android app, even from official documentation by Google. Here, we are going to give you a short description and then break down into its layers.

Clean Architecture is an architectural pattern that promotes the organization of code into layers with well-defined responsibilities, making the application more modular, maintainable, testable, and scalable. The key benefits of using Clean Architecture are as follows:

- **Separation of concerns (SoC)**: Clean Architecture organizes code into distinct layers with specific responsibilities, ensuring that each layer handles a separate aspect of the application. This SoC leads to a more modular and maintainable code base, making it easier to understand, modify, and extend.
- **Testability**: By separating the different concerns into independent layers, it becomes easier to test each layer in isolation. This allows developers to write comprehensive unit and integration tests, ensuring that the application behaves correctly and is less prone to bugs.
- **Reusability**: The modular structure of Clean Architecture promotes reusability by encouraging the creation of components that can be easily shared across different parts of the application or even between different projects. This reduces code duplication and improves the overall efficiency of the development process.

- **Flexibility**: Clean Architecture decouples the various layers of the application, making it easier to change or update any of these layers independently without affecting the others. This provides more flexibility when refactoring, making changes to the application, or adapting to new requirements.
- **Scalability**: The modular nature of Clean Architecture makes it easier to scale the application as it grows in complexity or size. By organizing code into well-defined layers and components, developers can more easily add new features, update existing functionality, or improve performance without introducing unintended side effects or making the code base unmanageable.
- **Easier collaboration**: Clean Architecture helps teams work more effectively by providing a clear structure and guidelines for organizing code. This makes it easier for developers to understand the code base, find the components they need, and contribute to the project more efficiently.
- **Future-proofing**: By adhering to the principles of Clean Architecture, you ensure that the application is built on a solid foundation that can evolve and adapt over time. This makes it more resilient to changes in technology, requirements, or team members, improving the long-term viability of the project.

In summary, using Clean Architecture in your projects leads to better-organized, more maintainable, and scalable code bases. It improves the overall quality of the application, reduces technical debt, and makes it easier for teams to work together effectively.

Now, with the benefits of Clean Architecture firmly in mind, let's delve into the specifics. What follows are the layers and the components of code that we will incorporate within each layer:

- **Presentation layer**:
  - **View**: This consists of UI components, such as **`Activity`**, **`Fragment`**, **`View`**, and, in our case, **`Composable`** components. The view is responsible for displaying data and capturing user input.

    - **ViewModel**: The `ViewModel` serves as a bridge between the `View` components and the data layers. It handles the UI logic, exposes `LiveData` or `StateFlow` objects for data binding, and communicates with `UseCase` classes.
- **Domain layer**:
    - **UseCase**: This layer contains the business logic and coordinates the flow of data between the data layer and the presentation layer. `UseCase` implementations encapsulate specific actions that can be performed within the app, such as sending a message, fetching chat history, or updating user settings.
- **Data layer**:
    - **Repository**: The `Repository` component is responsible for managing the data flow and providing a clean API to request data from different sources (local database, remote API, and so on). It abstracts the underlying data sources and handles caching, synchronization, and data merging.
    - **Data source**: This layer contains the implementations for accessing specific data sources such as local databases (using Room or another **object-relational mapper** (**ORM**)) and remote APIs (using Retrofit or another networking library, as in our case where we are using Ktor).

In the following diagram, we can see the relationships between the different layers and the typical components every layer includes:

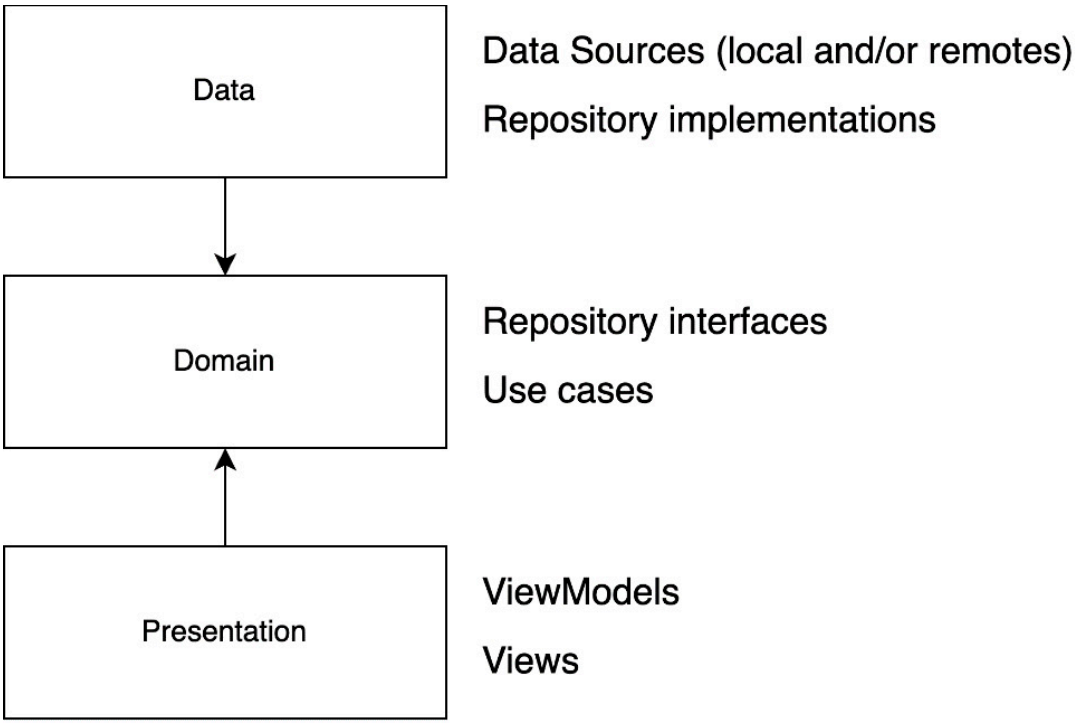

Figure 2.1: Clean Architecture in Android with the typical components per layer

Having this clear understanding of Clean Architecture's benefits and structure, let's now put these principles into practice.

Creating our Clean Architecture components

We have started building the data layer components, where we have created the **`MessagesWebsocket DataSource`** component. Now, it is time we build the rest of our Clean Architecture layers and components to reach the presentation layer.

In the end, this is what our app's Clean Architecture layers and components should look like:

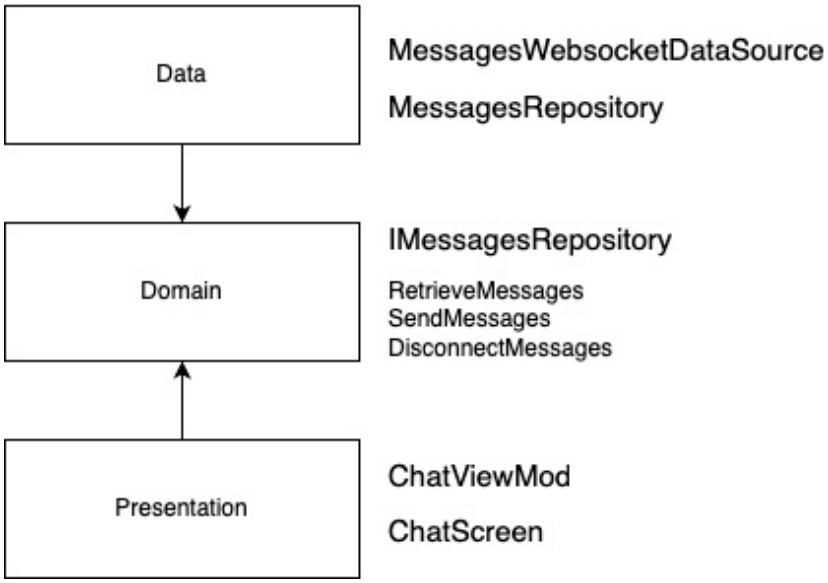

Figure 2.2: Layers and components that we will build in our project, following Clean Architecture principles

As we have already built the **`MessagesWebsocketDataSource`** component, the next component is the repository. The repository component will only connect with **`MessagesWebsocketDataSource`** (for now; we have bigger plans for it in the next chapter). We are going to call it **`MessagesRepository`**. Let's start building it:

```
class MessagesRepository @Inject constructor(
    private val dataSource: MessagesSocketDataSource
) {
    suspend fun getMessages(): Flow<Message> {
        return dataSource.connect()
    }
    suspend fun sendMessage(message: Message) {
        dataSource.sendMessage(message)
    }
    suspend fun disconnect() {
        dataSource.disconnect()
    }
}
```

`MessagesRepository` will just have one dependency (**`MessagesSocketDataSource`**) and will implement the functionality to connect to messages (the **`getMessages`** function), send messages (the

`sendMessage` function), and disconnect from the WebSocket (the **`disconnect`** function).

Now, we need to do a little modification to **`MessagesRepository`**: we need to create an interface in the domain layer with the **`MessagesRepository`** functionality. Creating an interface for the repository in the domain layer and implementing it in the data layer is a technique that follows the **Dependency Inversion Principle** (**DIP**) from the SOLID principles of **object-oriented programming** (**OOP**).

NOTE

DIP is one of the five principles of OOP and design known as SOLID. DIP states that high-level modules should not depend on low-level modules and both should depend on abstractions. Similarly, abstractions should not depend on details; details should depend on abstractions. The main idea behind DIP is to decouple modules, classes, or components in a software system, promoting flexibility, reusability, and maintainability. By depending on abstractions rather than concrete implementations, the system becomes more adaptable to changes and easier to test and maintain.

Let's create our **`IMessagesRepository`** interface:

```
interface IMessagesRepository {
    suspend fun getMessages(): Flow<Message>
    suspend fun sendMessage(message: Message)
    suspend fun disconnect()
}
```

Then, we will change our **`MessagesRepository`** class to implement this interface, adding the override in its functions:

```
class MessagesRepository @Inject constructor(
    private val dataSource: MessagesSocketDataSource
): IMessagesRepository {
```

```
    override suspend fun getMessages(): Flow<Message> {
        return dataSource.connect()
    }
    override suspend fun sendMessage(message: Message) {
        dataSource.sendMessage(message)
    }
    override suspend fun disconnect() {
        dataSource.disconnect()
    }
}
```

Now, we will continue in our journey to the presentation layer, implementing the domain layer.

The domain layer is not strictly mandatory, but it is highly recommended. While you can eliminate the domain layer and directly use repositories in your **`ViewModel`** instances, doing so would mix the responsibilities of the layers, which can lead to more complex and harder-to-maintain code. There may be cases where not implementing it could be considered; for example, if you are doing a **proof of concept** (**PoC**) or a simple app. Therefore, it is recommended to include the **`UseCase`** layer to maintain a clean and scalable architecture.

Following the **Single Responsibility Principle** (**SRP**), in this layer, we will create as many **`UseCase`** instances as different functions/responsibilities in our business logic. So, in our case, we will create three **`UseCase`** instances: one for retrieving messages, one for sending messages, and one for disconnecting or stopping message retrieval.

NOTE

SRP is one of the five principles of OOP and design known as SOLID. It states that a class, module, or function should have only one reason to change, meaning it should have only one responsibility. The principle aims to promote SoC by encouraging developers to break down their code into smaller, more focused components that handle a single task or as-

pect of the application. This leads to a more modular, maintainable, and easier-to-understand code base.

First, we will implement the **`RetrieveMessages`** use case:

```
class RetrieveMessages @Inject constructor(
    private val repository: IMessagesRepository
) {
    suspend operator fun invoke(): Flow<Message> {
        return repository.getMessages()
    }
}
```

Here, we have just one dependency: the repository. Note that we are declaring it using its interface. It is relevant because, as we detailed before, the domain shouldn't know anything about the data layer.

`RetrieveMessages` will have one function that will return a **`Flow`** instance with **`Message`** objects. For doing that, it will return **`repository.getMessages()`**. No mapping or alteration is needed as this function already returned a **`Flow`** instance of **`Message`** objects.

Second, we are going to implement the **`SendMessage`** use case:

```
class SendMessage @Inject constructor(
    private val repository: IMessagesRepository
) {
    suspend operator fun invoke(message: Message) {
        repository.sendMessage(message)
    }
}
```

Again, this use case will depend only on the **`IMessagesRepository`** interface. It will call its **`sendMessage`** function.

And finally, we will code the **`DisconnectMessages`** use case:

```
class DisconnectMessages @Inject constructor(
    private val repository: IMessagesRepository
) {
    suspend operator fun invoke() {
        repository.disconnect()
    }
}
```

The **`DisconnectMessages`** use case, as with the previous use cases implemented, depends on the **`IMessagesRepository`** interface and will call its **`disconnect`** function.

We are now done with the domain layer. Now, it's time to implement the **`ViewModel`** component that we will connect to the **`ChatScreen`** component, using **`ChatViewModel`**.

Implementing ChatViewModel

In Android, **`ViewModel`** is an architectural component introduced as part of the **Android Architecture Components** (**AAC**) library. It is designed to store and manage UI-related data in a lifecycle-conscious way. The primary responsibility of a **`ViewModel`** component is to hold and process the data required for a UI component (such as an **`Activity`**, **`Fragment`**, or **`Composable`** component) while properly handling configuration changes (such as device rotations) and surviving the lifecycle of the associated UI component.

Our **`ChatViewModel`** class will be responsible for handling the data required in our **`ChatScreen`** component (which we previously built in ***Chapter 1***). This data will come and change from the use cases we have just created. So first, our **`ChatViewModel`** class will have those use cases as dependencies:

```
@HiltViewModel
class ChatViewModel @Inject constructor(
```

```
    private val retrieveMessages: RetrieveMessages,
    private val sendMessage: SendMessage,
    private val disconnectMessages: DisconnectMessages
) : ViewModel() {
// ....
}
```

Then, we will need a property to hold the state. This property needs to be observable from the view but read-only (so that the view shouldn't be able to modify it). We will solve this by creating two different properties. The first property is **`_messages`**:

```
private val _messages =
MutableStateFlow<List<Message>>(emptyList())
```

This line creates a private mutable state flow that holds a list of **`Message`** objects. We will use it to manage and update messages internally within the **`ViewModel`**.

The second property will be **`messages`**:

```
val messages: StateFlow<List<Message>> = _messages
```

This line exposes the private mutable state flow as a public read-only state flow. This allows the UI components to observe messages without being able to modify them directly.

Now, we need to implement the **`loadAndUpdateMessages`** function that will call the **`RetrieveMessages`** use case:

```
private var messageCollectionJob: Job? = null
fun loadAndUpdateMessages() {
    messageCollectionJob =
    viewModelScope.launch(Dispatchers.IO) {
        retrieveMessages()
            .map { it.toUI() }
```

```
            .collect { message ->
                withContext(Dispatchers.Main) {
                    _messages.value = _messages.value +
                    message
                }
            }
    }
}
```

In the previous code block, it can be seen that we need to declare a **`messageCollectionJob`** variable. This variable is used to cancel the **`messages`** collection job when the **`ViewModel`** is cleared.

The **`loadAndUpdateMessages`** function is responsible for fetching and updating messages. It launches a coroutine with the **`Dispatchers.IO`** context for performing network or disk operations.

Inside the coroutine, the **`retrieveMessages`** function is called, and the resulting messages are mapped into the **`Message`** UI object and then collected using the **`collect`** function.

For each collected message, the **`_messages state`** flow is updated with the new message by switching the coroutine context to **`Dispatchers.Main`**.

Next, to make the mapping more readable, we are going to create two extension functions:

```
private fun DomainMessage.toUI(): Message {
    return Message(
        id = id,
        senderName = senderName,
        senderAvatar = senderAvatar,
        timestamp = timestamp,
        isMine = isMine,
        messageContent = getMessageContent()
    )
```

```
}
private fun DomainMessage.getMessageContent():
MessageContent {
    return when (contentType) {
        DomainMessage.ContentType.TEXT ->
            MessageContent.TextMessage(content)
        DomainMessage.ContentType.IMAGE ->
            MessageContent.ImageMessage(content,
            contentDescription)
    }
}
```

So, when retrieving and mapping messages, we just have to call the following:

```
retrieveMessages()
    .map { it.toUI() }
```

Then, we continue to process the **`messages`** collection job.

Then, we should add a function to send a new message. Basically, the idea is to launch the coroutine in the **`Dispatchers.IO`** context to send the message. As it is a network operation, it is recommended to use the I/O dispatcher and map the **`String`** object we are getting from the user to the domain object, as you can see in the following code block:

```
fun onSendMessage(messageText: String) {
    viewModelScope.launch(Dispatchers.IO) {
        val message = Message(messageText) // We will add
                                           here the rest
                                           of the fields
        sendMessage(message)
    }
}
```

Note that, to create the domain object, we are going to lack some information because, for example, we have no way to obtain the **`senderImage`** or the **`senderName`** properties that are mandatory to send a message.

So, this function is not going to compile for now, but we will solve this problem in the following section.

Finally, we can use the **`onCleared`** function to disconnect from the message's retrieval:

```
override fun onCleared() {
    messageCollectionJob?.cancel()
    viewModelScope.launch(Dispatchers.IO) {
        disconnectMessages()
    }
}
```

This function is called when the **`ViewModel`** is no longer in use and will be disposed of by the system. This involves canceling the **`messageCollectionJob`** variable, provided it's not **`null`**, effectively halting the **`messages`** collection coroutine. Concurrently, in the context of **`Dispatchers.IO`**, a new coroutine is launched to execute the **`disconnectMessages`** function. This guarantees that any essential cleanup associated with disconnecting from the message source is carried out properly.

This is how the **`ChatViewModel`** component will look (for now):

```
import com.packt.feature.chat.domain.models.Message as
DomainMessage
// We are using this import with an alias to make it easier
   to identify the Message class from the domain layer
@HiltViewModel
class ChatViewModel @Inject constructor(
    private val retrieveMessages: RetrieveMessages,
    private val sendMessage: SendMessage,
    private val disconnectMessages: DisconnectMessages
) : ViewModel() {
    private val _messages =
        MutableStateFlow<List<Message>>(emptyList())
```

```
    val messages: StateFlow<List<Message>> = _messages
    private var messageCollectionJob: Job? = null
    fun loadAndUpdateMessages() {
        messageCollectionJob =
        viewModelScope.launch(Dispatchers.IO) {
            retrieveMessages()
                .map { it.toUI() }
                .collect { message ->
                    withContext(Dispatchers.Main) {
                        _messages.value = _messages.value +
                        message
                }
            }
        }
    }
    private fun DomainMessage.toUI(): Message {
        return Message(
            id = id,
            senderName = senderName,
            senderAvatar = senderAvatar,
            timestamp = timestamp,
            isMine = isMine,
            messageContent = getMessageContent()
        )
    }
    private fun DomainMessage.getMessageContent():
    MessageContent {
        return when (contentType) {
            DomainMessage.ContentType.TEXT ->
                MessageContent.TextMessage(content)
            DomainMessage.ContentType.IMAGE ->
                MessageContent.ImageMessage(content,
                contentDescription)
        }
    }
    fun onSendMessage(messageText: String) {
        viewModelScope.launch(Dispatchers.IO) {
            val message = Message(messageText)
            sendMessage(message)
        }
    }
```

```
    override fun onCleared() {
        messageCollectionJob?.cancel()
        viewModelScope.launch(Dispatchers.IO) {
            disconnectMessages()
        }
    }
}
```

Now that we have our **`ChatViewModel`** component ready, we need to connect it to the view. We will make the changes needed in the **`ChatScreen`** component so that it connects to our **`ChatViewModel`** component. As the first step, we have added the **`ViewModel`** to the arguments:

```
@Composable
fun ChatScreen(
    viewModel: ChatViewModel = hiltViewModel(),
    chatId: String?,
    onBack: () -> Unit
) {
}
```

Then, we will also add a **`LaunchEffect`** composable that will start the messages' load:

```
LaunchedEffect(Unit) {
    viewModel.loadAndUpdateMessages()
}
```

Next, the **`SendMessageBox`** composable takes a lambda parameter, where we are going to send the message using the **`ViewModel`** function:

```
SendMessageBox { viewModel.onSendMessage(it) }
```

After that, we add the following new parameter to the **`SendMessageBox`** composable definition and call it in its **`IconButton`** **`onClick`** property:

```
@Composable
fun SendMessageBox(sendMessage: (String)->Unit) {
    Box(modifier = Modifier
        .defaultMinSize()
        .padding(top = 0.dp, start = 16.dp, end = 16.dp,
            bottom = 16.dp)
        .fillMaxWidth()
    ) {
        var text by remember { mutableStateOf("") }
        OutlinedTextField(
            value = text,
            onValueChange = { newText -> text = newText },
            modifier = Modifier
                .fillMaxWidth(0.85f)
                .align(Alignment.CenterStart)
                .height(56.dp),
        )
        IconButton(
            modifier = Modifier
                .align(Alignment.CenterEnd)
                .height(56.dp),
            onClick = {
                sendMessage(text)
                text = ""
            }
        ) {
            Icon(
                imageVector = Icons.Default.Send,
                tint = MaterialTheme.colors.primary,
                contentDescription = "Send message"
            )
        }
    }
}
```

Finally, we will inject the **`messages`** property to the **`ListOfMessages`** composable:

```
ListOfMessages(paddingValues = paddingValues, messages = messages)
```

This, of course, will also require a change in the composable definition and code:

```
@Composable
fun ListOfMessages(messages: List<Message>, paddingValues: PaddingValues) {
    Box(modifier = Modifier
        .fillMaxSize()
        .padding(paddingValues)) {
        Row(modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp)
        ) {
            LazyColumn(
                modifier = Modifier
                    .fillMaxSize(),
                verticalArrangement =
                    Arrangement.spacedBy(8.dp),
            ) {
                items(messages) { message ->
                    MessageItem(message = message)
                }
            }
        }
    }
}
```

Instead of using the **`getFakeMessages()`** function we were using when we built the **`ListOfMessages`** composable, we will use the **`messages`** list that we are now obtaining via properties.

And with that, we've covered almost everything, but there remain some challenges to address. For instance, we don't have the necessary information to display the correct avatar and name of the chat members or the necessary information to fill in the required properties for sending a message. While we will receive new messages once we connect to the

WebSocket, the question of how to get historical messages remains. We will tackle these issues, along with other concerns related to error handling and synchronization, in the upcoming section.

Handling synchronization and errors

To make the chat messages functionality complete, we still have some issues we have to take into account: getting historical messages and receiver information and handling possible errors. We will go through them in this section.

Obtaining chat screen initialization data

Apart from the messages that we are going to be receiving or sending via the data source, we still need to get some additional information. This includes the following:

- Messages that have been sent and received before the WebSocket was connected (not all of them, though, because the conversation could have many messages, and it would take a long time to gather/load all of the information; instead we should prioritize fetching a certain number of the most recent messages)
- Receiver information, such as their name or avatar URL

There are several options to solve this – for example, we could have a different type of message with all this information when the WebSocket connection is established, or we could have a specific API call to retrieve this information. As we have already played with the Ktor WebSocket for the chat feature, we are going to use it to implement an API call to retrieve this information.

When we built **`WebsocketMessagesDataSource`**, we had to provide an **`HttpClient`** instance. Usually, these clients are shared within the same

application, but we should create a new one to be used for our API requests. For that, we would need to add a new dependency:

```
implementation "io.ktor:ktor-client-content-negotiation:
$ktor_version"
```

Then, we can create the client like so (we can do it in the same file we defined the WebSocket client):

```
object RestClient {
    val client = HttpClient{
        install(ContentNegotiation) {
            json()
        }
    }
}
```

Next, we are going to create a **`ChatRoomDataSource`** class that will be in charge of handling this data retrieval:

```
class ChatRoomDataSource @Inject constructor(
    private val client: HttpClient,
    private val url: String
) {
    suspend fun getInitialChatRoom(id: String):
    ChatRoomModel {
        return client.get(url.format(id)).body()
    }
}
```

As seen here, we are going to inject the client and the URL as dependencies. Then, in the **`getInitialChatRoom`** function, we will call the **`client.get(url)`** function in order to make a request to the endpoint.

Using the Ktor client, you can use various HTTP methods. Here's a list of common ones:

- **`GET`**: Retrieves data from the specified endpoint. To use this method in Ktor, you can call the **`get`** function:

```
val response: HttpResponse =
client.get("https://api.example.com/data")
```

- **`POST`**: Sends data to the specified endpoint, usually for creating a new resource. To use this method in Ktor, you can call the **`post`** function:

```
val response: HttpResponse =
client.post("https://api.example.com/data") {
body = yourData }
```

- **`PUT`**: Sends data to the specified endpoint, usually for updating an existing resource. To use this method in Ktor, you can call the **`put`** function:

```
val response: HttpResponse =
client.put("https://api.example.com/data") {
body = yourUpdatedData }
```

- **`DELETE`**: Deletes a specified resource. To use this method in Ktor, you can call the **`delete`** function:

```
val response: HttpResponse =
client.delete("https://api.example.com/data/ID")
```

- **`PATCH`**: Applies partial modifications to a resource. To use this method in Ktor, you can call the **`patch`** function:

```
val response: HttpResponse =
client.patch("https://api.example.com/data") {
body = yourPartialData }
```

In the case of our **`getInitialChatRoom`** function, we are using the **`client.get(URL)`** function (note that we have to provide the URL in a

format such that we can then replace the ID of **`ChatRoom`**). We also need to return a new model, **`ChatRoomModel`**:

```
@kotlinx.serialization.Serializable
data class ChatRoomModel(
    val id: String,
    val senderName: String,
    val senderAvatar: String,
    val lastMessages: List<WebsocketMessageModel>
)
```

Now, in order to provide the dependencies that **`ChatRoomDataSource`** needs, we have to set our **`ChatModule`** class in the following way:

```
@InstallIn(SingletonComponent::class)
@Module
abstract class ChatModule {
    companion object {
        const val WEBSOCKET_URL =
            "ws://whatspackt.com/chat/%s"
        const val WEBSOCKET_URL_NAME = "WEBSOCKET_URL"
        const val WEBSOCKET_CLIENT = "WEBSOCKET_CLIENT"
        const val API_CHAT_ROOM_URL =
            "http://whatspackt.com/chats/%s"
        const val API_CHAT_ROOM_URL_NAME = "CHATROOM_URL"
        const val API_CLIENT = "API_CLIENT"
    }
    @Provides
    @Named(WEBSOCKET_CLIENT)
    fun providesWebsocketHttpClient(): HttpClient {
        return WebsocketClient.client
    }
    @Provides
    @Named(WEBSOCKET_URL_NAME)
    fun providesWebsocketURL(): String {
        return WEBSOCKET_URL
    }
    @Binds
```

```
    abstract fun providesMessagesRepository(
        messagesRepository: MessagesRepository
    ): IMessagesRepository
    @Provides
    @Named(API_CLIENT)
    fun providesAPIHttpClient(): HttpClient {
        return RestClient.client
    }
}
```

As both the **`providesWebsocketClient`** and **`providesApiHttpClient`** functions are returning the same type (**`HttpClient`**), we need them to be identifiable so that we can indicate to Hilt which dependency it should provide to **`WebsocketDataSource`** and which one goes to **`ChatRoomDataSource`**. That's the reason we are using qualifiers.

NOTE

Using qualifiers allows the **dependency injection** *(***DI***) framework to determine the correct instance of a dependency to inject when there are multiple instances available of the same type. This ensures that the right instance is provided, preventing conflicts or ambiguity in your dependency management.*

In the next code block, we are using a **`WEBSOCKET_CLIENT`** constant as the qualifier for the WebSocket **`HttpClient`** instance and **`API_CLIENT`** for the REST API **`HttpClient`** instance:

```
@Provides
@Named(WEBSOCKET_CLIENT)
fun providesWebsocketHttpClient(): HttpClient {
    return WebsocketClient.client
}
@Provides
@Named(API_CLIENT)
fun providesAPIHttpClient(): HttpClient {
```

```
        return RestClient.client
    }
```

We should also use qualifiers to provide URLs for the WebSocket and for the API. Also, it is important to note that these URL values are now being provided by a companion object in **`ChatModule`** for simplification, but a better approach would be to have them defined as part of our Gradle file. That way, we will be able to override them depending on the build variant (release, debug, test, and so on) or flavor.

Regarding the qualifiers, we also need to indicate in the consumers of these dependencies which one should be injected. This will be done using the **`@Named`** annotation in the affected dependencies as follows:

```
class ChatRoomDataSource @Inject constructor(
    @Named(API_CLIENT) private val client: HttpClient,
    @Named(API_CHAT_ROOM_URL_NAME) private val url: String
) {
    suspend fun getInitialChatRoom(id: String):
    ChatRoomModel {
        return client.get(url.format(id)).body()
    }
}
```

Also, we have to modify the constructor in **`MessagesSocketDataSource`** so that Hilt knows which one it has to inject:

```
class MessagesSocketDataSource @Inject constructor(
    @Named(WEBSOCKET_CLIENT) private val httpClient:
        HttpClient,
    @Named(WEBSOCKET_URL_NAME) private val websocketUrl:
        String
) { ... }
```

Now that we have everything ready for our dependencies to be injected the correct way, it is time to implement the **`ChatRoomRepository`** compo-

nent. We will implement it in a similar way that we implemented the **`MessagesRepository`** component.

First, we want to create an interface in our domain package:

```
package com.packt.feature.chat.domain
import com.packt.feature.chat.domain.models.ChatRoom
interface IChatRoomRepository {
    suspend fun getInitialChatRoom(id: String): ChatRoom
}
```

Then, we will create the actual implementation in the **`data.repository`** package:

```
package com.packt.feature.chat.data.network.repository
import com.packt.feature.chat.data.network.datasource
.ChatRoomDataSource
import com.packt.feature.chat.domain.IChatRoomRepository
import com.packt.feature.chat.domain.models.ChatRoom
import javax.inject.Inject
class ChatRoomRepository @Inject constructor(
    private val dataSource: ChatRoomDataSource
): IChatRoomRepository {
    override suspend fun getInitialChatRoom(id: String):
    ChatRoom {
        val chatRoomApiModel =
            dataSource.getInitialChatRoom(id)
        return chatRoomApiModel.toDomain()
    }
}
```

Here, we are obtaining the initial chat room information from the data source, and then we will map the obtained data model into the domain model.

Of course, this will not work unless we create the domain model, **`ChatRoom`**:

```
package com.packt.feature.chat.domain.models
data class ChatRoom(
    val id: String,
    val senderName: String,
    val senderAvatar: String,
    val lastMessages: List<Message>
)
```

Then, we should create the mapping from **`ChatRoomModel`**:

```
@Serializable
data class ChatRoomModel(
    val id: String,
    val senderName: String,
    val senderAvatar: String,
    val lastMessages: List<WebsocketMessageModel>
) {
    fun toDomain(): ChatRoom {
        return ChatRoom(
            id = id,
            senderName = senderName,
            senderAvatar = senderAvatar,
            lastMessages = lastMessages.map { it.toDomain() }
        )
    }
}
```

Here, we have just added the **`toDomain()`** function, which will map the data object (**`ChatRoomModel`**) to the domain object (**`ChatRoom`**).

Now, we need to bind the repository interface to its implementation. For that, we should add a binding declaration to our Hilt module:

```
@Binds
abstract fun providesChatRoomRepository(
```

```
    chatRoomRepository: ChatRoomRepository
): IChatRoomRepository
```

Here, we are saying to Hilt that every time it needs to provide an **`IChatRoomRepository`** dependency, it should provide **`ChatRoomRepository`**.

Now, we have the data source and the repository ready. We will need to implement a new use case whose responsibility will be to provide this initial information:

```
package com.packt.feature.chat.domain.usecases
import com.packt.feature.chat.domain.IChatRoomRepository
import com.packt.feature.chat.domain.models.ChatRoom
import javax.inject.Inject
class GetInitialChatRoomInformation @Inject constructor(
    private val repository: IChatRoomRepository
) {
    suspend operator fun invoke(id: String): ChatRoom {
        return repository.getInitialChatRoom(id)
    }
}
```

Here, we will be calling the repository **`getInitialChatRoom()`** function, to obtain it in the **`ChatRoom`** model.

We are now arriving at our destination: the **`ViewModel`**. We need to include **`GetInitial ChatRoomInformation`** as a dependency on the **`ViewModel`**, obtain this information when it is initialized, and make it available for the UI to observe it:

```
@HiltViewModel
class ChatViewModel @Inject constructor(
    private val retrieveMessages: RetrieveMessages,
    private val sendMessage: SendMessage,
    private val disconnectMessages: DisconnectMessages,
    private val getInitialChatRoomInformation:
```

```
        GetInitialChatRoomInformation
) : ViewModel() {...}
```

Next, we need to create a new **`StateFlow`** instance to be consumed by the UI. As it is going to hold the state of almost all the UI (except the messages; we will talk about this later), we are going to call it **`uiState`**:

```
private val _uiState = MutableStateFlow(Chat())
val uiState: StateFlow<Chat> = _uiState
```

Now, we are going to add a new function to be called upon view initialization:

```
fun loadChatInformation(id: String) {
    messageCollectionJob =
    viewModelScope.launch(Dispatchers.IO) {
        val chatRoom = getInitialChatRoomInformation(id)
        withContext(Dispatchers.Main) {
            _uiState.value = chatRoom.toUI()
            _messages.value = chatRoom.lastMessages.map {
                it.toUI()}
            updateMessages()
        }
    }
}
```

Here, we are using **`messagesCollectionJob`** (we could change its name to make it more generic as now it is going to be used by the **`messages`** collection job and the initial data retrieval).

Then, we retrieve the initial chat room information, update the **`uiState`** value, and set the messages we are receiving as the first messages in the **`messages StateFlow`** object (so that the chat will show the old messages).

Finally, we call the **`updateMessages()`** function, where we will connect to the WebSocket and start getting asynchronous messages.

Note that we will also need a **`Chat`** model that will be our **`uiState`** instance; this model is important as it will be the object consumed from the UI to configure it. Add this like so:

```
data class Chat(
    val id: String? = null,
    val name: String? = null,
    val avatar: String? = null
)
fun ChatRoom.toUI() = run {
    Chat(
        id = id,
        name = senderName,
        avatar = senderAvatar
    )
}
```

Now, we need to listen to this **`uiState`** instance from our screen composable and update the UI accordingly:

```
@Composable
fun ChatScreen(
    viewModel: ChatViewModel = hiltViewModel(),
    chatId: String?,
    onBack: () -> Unit
) {
    val messages by viewModel.messages.collectAsState()
    val uiState by viewModel.uiState.collectAsState()
    LaunchedEffect(Unit) {
        viewModel.loadChatInformation(chatId.orEmpty())
    }
    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                   Text(stringResource(R.string.chat_title,
                   uiState.name.orEmpty()))
                }
```

```
                )
            },
            bottomBar = {
                SendMessageBox { viewModel.onSendMessage(it) }
            }
        ) { paddingValues->
            ListOfMessages(paddingValues = paddingValues,
            messages = messages)
        }
    }
```

Here, we can see that we are calling the **`loadChatInformation`** function as soon as the **`Composable`** component is started. Then, once this information is obtained, we would show the name of the participant of the chat in the **`TopAppBar`** component, obtaining this info from the chat initialization. At the same time, the list of messages will be updated with the last messages.

Usually, it is desirable to encapsulate all the **`uiState`** properties in a single observable value as one of the advantages of Jetpack Compose is that it will handle the recomposition when it detects that the values related to a **`Composable`** component have changed. In this case, the criteria followed have been to separate them because in reality, the frequency of changes is very different between the two values:

- The **`uiState`** properties are not going to change for the same chat
- The **`messages`** list is likely to change with a high frequency (every time we send and receive a message)

During this section, we have set up our chat initialization, including all the components needed for the architecture, from the data source to the **`ViewModel`** changes. Now, it is time we take care of possible errors we could encounter and give some resilience to our chat screen.

Handling errors in the WebSocket

Errors are not unusual, especially in a long-lived connection such as a WebSocket, and in such a sensitive environment as a mobile one, it is important to take care of these errors because otherwise, our users could stop being able to send or receive messages and, in the worst case, have a fatal error that crashes the application.

There are several ways we can control these errors. One of them is to make every layer responsible for errors that could happen in its scope and only propagate to the UI (or the user knowledge) when the app cannot recover itself from them.

Here, we could have several errors:

- Connection errors that are recuperable errors and will be handled by a retry
- Parsing errors that are likely not recuperable as several retries will not change the way the app or the backend are formatting the messages (we cannot do much with these kinds of errors, apart from detecting them before deploying the app or having analytics tools to detect them)

In this section, we are going to focus on **`MessagesSocketDataSource`**. If we take a look at our **`connect`** function, we can see it could have some points of failure (for example, when initiating the session or when the message received is handled). The simplest way to solve this is to wrap those points with **`try-catch`** blocks:

```
suspend fun connect(): Flow<Message> {
    return flow {
        // Wrap the connection attempt with a try-catch
           block
        try {
            httpClient.webSocketSession { url(websocketUrl) }
                .apply { webSocketSession = this }
                .incoming
                .receiveAsFlow()
```

```
                .collect { frame ->
                    try {
                        // Handle errors while processing
                           the message
                        val message =
                            webSocketSession.handleMessage(
                                frame)?.toDomain()
                        if (message != null) {
                            emit(message)
                        }
                    } catch (e: Exception) {
                        // Log or handle the error
                           gracefully
                        Log.e(TAG, "Error handling
                            WebSocket frame", e)
                    }
                }
        } catch (e: Exception) {
            // Log or handle the connection error
               gracefully
            Log.e(TAG, "Error connecting to WebSocket", e)
        }
    }.retryWhen { cause, attempt ->
        // Implement a retry strategy based on the cause
           and/or the number of attempts
        if (cause is IOException && attempt < MAX_RETRIES)
        {
            delay(RETRY_DELAY)
            true
        } else {
            false
        }
    }.catch { e ->
        // Handle exceptions from the Flow
        Log.e(TAG, "Error in WebSocket Flow", e)
    }
}
```

We need to define also as constants **`TAG`** (to log messages in Logcat), **`MAX_RETRIES`**, which will be the number of retries we are going to use

(because we cannot be eternally retrying), and **`RETRY_DELAY`** (the milliseconds we are going to wait between retries):

```
companion object {
    const val TAG = "MessagesSocketDataSource"
    const val RETRY_DELAY = 30000
    const val MAX_RETRIES = 5
}
```

Here, we are defining these values as constants, so if the WebSocket connection fails, we will retry the connection in another 30 seconds (**`30000`** milliseconds). This will occur **`5`** times before giving up if it doesn't successfully connect.

Now that our users are receiving messages while using the app, we still need to provide a way of notifying them when they receive a new message but are not using the app. We can solve this problem by using push notifications.

Adding push notifications

Push notifications are messages that are sent to a user's device from a server, even when the user is not actively using the app. These messages appear as system notifications outside of the app and can be used to provide updates, alerts, or other relevant information to users.

To send push notifications, we need to decide which of the available options we want to use. The most popular is **Firebase Cloud Messaging** (**FCM**), but there are more push notification services such as OneSignal, Pusher, or **Amazon Simple Notification Service** (**SNS**). In our case, we are going to take the popular route and use FCM.

Firebase is a mobile and web application development platform provided by Google. It offers a suite of tools, services, and infrastructure designed to help developers build, improve, and grow their apps. Some of

its features include authentication, push notifications, cloud databases, and so on. We are going to use it for the last two sections of this chapter.

To accomplish that, we first need to set up Firebase in our project.

Setting up Firebase

To set up Firebase in our project, we need to follow these steps:

1. Go to the Firebase console (**https://console.firebase.google.com/**) and click **Add project**. Then, follow the onscreen instructions to set up your project.
2. In the Firebase console, click on the Android icon to register your app. Enter your app's package name, and optionally, provide the SHA-1 fingerprint for Google Sign-In and other authentication features. Click **Register app** to proceed.
3. After registering our app, we'll be prompted to download a **`google-services.json`** file. Download it and place it in the **`app`** module of our Android project, at the root level.
4. Add Firebase SDK dependencies to your project's **`build.gradle`** files, like so:

```
classpath 'com.google.gms:google-services:
$latest_version'
```

5. Then in the **`app`** module's **`build.gradle`** file where we are going to use it (in our case, **`:common:data`**), we should add these dependencies for the following specific Firebase services:

```
implementation platform('com.google.firebase:
    firebase-bom:$latest_version')
implementation 'com.google.firebase:firebase-auth'
implementation 'com.google.firebase:
    firebase-firestore'
implementation 'com.google.firebase:
    firebase-messaging'
```

Note that, as we did with Jetpack Compose dependencies, here we are going to use the **Bill of Materials** (**BoM**). The advantage is that we don't need to specify the version of every dependency because the compatible ones will be provided by the BoM.

NOTE

A BoM is a mechanism used in dependency management systems to specify and manage the versions of multiple libraries and their transitive dependencies as a single entity. It helps simplify dependency management and ensures compatibility between different libraries that are part of the same ecosystem or suite.

6. Also, in order to facilitate the use of coroutines to handle Firebase tasks, we are going to add this extra dependency:

```
implementation 'org.jetbrains.kotlinx:
kotlinx-coroutines-play-services:$latest_version'
```

Now, before we can receive a push notification, we need to identify our user. We do that by sending their token to Firebase.

Sending the FCM token to Firebase

To identify our users and send notifications specifically to them using FCM, we need to use FCM **tokens**. Each user is assigned a unique FCM token, which is used to send notifications to their devices. This token should be obtained and updated every time the user signs in or when the app starts.

We can obtain the FCM token by calling the **`getToken()`** method from the **`FirebaseMessaging`** class. To do that, we are going first to create a data source that will wrap the token-handling functionality:

```
package com.packt.data
import com.google.firebase.messaging.FirebaseMessaging
import kotlinx.coroutines.tasks.await
import javax.inject.Inject
class FCMTokenDataSource @Inject constructor(
    private val firebaseMessaging: FirebaseMessaging =
    FirebaseMessaging.getInstance()
) {
    suspend fun getFcmToken(): String? {
        return try {
            FirebaseMessaging.getInstance().token.await()
        } catch (e: Exception) {
            null
        }
    }
}
```

Here, we are injecting the **`FirebaseMessaging`** instance and obtaining the FCM token from Firebase.

Now, we need this FCM to be stored somewhere so that when a new message is sent to our users, we know which token is associated with them. There is no standard way to store it. Usually, this will be handled in the backend, which is far from the scope of this book. But we can prepare the app components needed. We are going to create a use case that would be the orchestrator of obtaining and then sending the FCM to be stored in the backend. This use case will need a repository to do both tasks: obtaining the token and storing it in our systems.

As always, create the interface for our repository in the domain layer (in this case, in the **`:common:domain`** module):

```
interface IFCMTokenRepository {
    suspend fun getFCMToken(): String
}
```

Then, we will create the repository implementation in the data layer (**`:common:data`**):

```
class FCMTokenRepository @Inject constructor(
    private val tokenDataSource: FCMTokenDataSource
) {
    suspend fun getToken(): String? {
        return tokenDataSource.getFcmToken()
    }
}
```

We will use this repository to obtain the token from Firebase. As said before, we also need to store the token somewhere, so we will create another repository for that:

```
interface IInternalTokenRepository {
    suspend fun storeToken(userId: String, token: String)
}
```

We will again leave the implementation empty as it is outside our scope. The relevant bit to understand here is that the token should be stored so that later, when our user receives a message, we can identify the token and send a push notification to the related device.

In the next code block, we can see how we are implementing the aforementioned interface, where you will provide the means to store the data source of your preference:

```
class InternalTokenRepository(): IInternalTokenRepository {
    override suspend fun storeToken(userId: String, token:
    String) {
        // Store in the data source of your choosing
    }
}
```

Now that we have the token sorted, we need to prepare our app to receive push notifications.

Preparing the app to receive push notifications

Push notifications are messages that pop up on a mobile device. They are especially useful when the user is not actively using the application and we need to call their attention. In this section, we are going to make our app capable of receiving them when a new message is received.

To start receiving push notifications, we need to make some modifications to our existing code first. For example, we have to think about what would we expect to happen if the user clicks on a notification: we may want it to open the **`ChatScreen`** component related to the message notification. Let's start with those changes.

To open the **`ChatScreen`** component directly, we will need to create a link that tells the system that it should open our application showing the **`ChatScreen`** component. This link is called a deep link.

A **deep link** is a type of link that directs a user to a specific piece of content or page within an Android application rather than just launching the application. Deep links are used to provide a more seamless user experience by allowing users to jump directly to a particular function, feature, or piece of content within an app from a website, another app, or even a simple text message or email.

To create our deep link, we are going to create an object called **`DeepLinks`** in the **`:common:framework`** module to organize all the deep links we are going to use in our application:

```
package com.packt.framework.navigation
object DeepLinks {
    const val chatRoute =
```

```
        "https://whatspackt.com/chat?chatId={chatId}"
}
```

Then, we need to modify our **`NavHost`** component– once the application receives an intent with this deep comlink, the app should navigate to the **`ChatScreen`** component. To accomplish that, we need to add a **`Deeplink`** instance as an option for the **`ChatScreen`** navigation graph in **`WhatsPacktNavigation`**:

```
private fun NavGraphBuilder.addChat(navController:
NavHostController) {
    composable(
        route = NavRoutes.Chat,
        arguments = listOf(
            navArgument(NavRoutes.ChatArgs.ChatId) {
                type = NavType.StringType }),
        deepLinks = listOf(
            navDeepLink {
                uriPattern = DeepLinks.chatRoute
            }
        )
    ) { backStackEntry ->
        val chatId = backStackEntry.arguments?.getString(
            NavRoutes.ChatArgs.ChatId)
        ChatScreen(chatId = chatId, onBack = {
            navController.popBackStack() })
    }
}
```

Here, we are adding the deep link pattern that we have in our **`DeepLinks`** object to be included as one of the route options for our **`ChatScreen`** component.

Then, we need to implement a **`FirebaseMessagingService`** function that will catch all the push notifications that we receive and will allow us to define a channel where notifications will be posted and handled by the Android system, ultimately showing them to the user (if the user has given our app permissions to do that):

```
class WhatsPacktMessagingService:
FirebaseMessagingService() {
    companion object {
        const val CHANNEL_ID = "Chat_message"
        const val CHANNEL_DESCRIPTION = "Receive a
            notification when a chat message is received"
        const val CHANNEL_TITLE = "New chat message
            notification"
    }
    override fun onMessageReceived(remoteMessage:
    RemoteMessage) {
        super.onMessageReceived(remoteMessage)
        if (remoteMessage.data.isNotEmpty()) {
            // We can extract information such as the
               sender, message content, or chat ID
            val senderName =
                remoteMessage.data["senderName"]
            val messageContent =
                remoteMessage.data["message"]
            val chatId = remoteMessage.data["chatId"]
            val messageId = remoteMessage.data["messageId"]
            // Create and show a notification for the
               received message
            if (chatId != null && messageId != null) {
                showNotification(senderName, messageId,
                messageContent, chatId)
            }
        }
    }
    private fun showNotification(senderName: String?,
    messageId: String, messageContent: String?,
    chatId: String) {
        // Implement here the notification
    }
}
```

Here, we are extracting some information from the message received, such as **`senderName`**, **`messageContent`**, **`chatId`**, and so on. Ideally, we could obtain the information we want to show in the notification.

This is just an example, though – the information structure would depend on the payload contract we already defined with the backend implementation.

Once we have extracted this information, we need to show the notification:

```
private fun showNotification(senderName: String?,
messageId: String, messageContent: String?, chatId: String)
{
    val notificationManager = getSystemService(
       Context.NOTIFICATION_SERVICE) as NotificationManager
    // Create a notification channel
    // (if you want to support versions lower than Android
       Oreo, you will have to check the version here)
    val channel = NotificationChannel(
        CHANNEL_ID,
        CHANNEL_TITLE,
        NotificationManager.IMPORTANCE_DEFAULT
    ).apply {
        description = CHANNEL_DESCRIPTION
    }
    notificationManager.createNotificationChannel(channel)
    // Create an Intent to open the chat when the
       notification is clicked. Here is where we are going
       to use our newly created deeplink
    val deepLinkUrl =
        DeepLinks.chatRoute.replace("{chatId}", chatId)
    val intent = Intent(Intent.ACTION_VIEW,
    Uri.parse(deepLinkUrl)).apply {
        flags = Intent.FLAG_ACTIVITY_NEW_TASK or
        Intent.FLAG_ACTIVITY_CLEAR_TASK
    }
    // Create a PendingIntent for the Intent
    val pendingIntent = PendingIntent.getActivity(this, 0,
        intent, PendingIntent.FLAG_IMMUTABLE)
    // Build the notification
    val notification = NotificationCompat.Builder(this,
        CHANNEL_ID)
```

```
            .setSmallIcon(R.drawable.our_notification_icon_for_
                whatspackt)
            .setContentTitle(senderName)
            .setContentText(messageContent)
            .setContentIntent(pendingIntent)
            .setAutoCancel(true)
            .build()
        // Show the notification
        notificationManager.notify(messageId.toInt(),
            notification)
    }
```

First, we create a **`NotificationChannel`** instance, then the elements we need for our notification (such as **`PendingIntent`**, which will be used when the user clicks on the notification), and then the notification itself (using **`NotificationCompat`**). Finally, we use **`NotificationManager`** to notify our notification to the system.

NOTE

Creating a **`NotificationChannel`** *instance is necessary for Android 8.0 (API level 26) and higher, as it provides users with better control over the app's notifications. Each* **`NotificationChannel`** *instance represents a unique category of notifications that an app can display, and users can modify the settings for each channel independently. This enables users to customize the behavior of your app's notifications based on their preferences.*

For example, users can set the importance level, enable/disable sound, or set a custom vibration pattern for each channel. They can also block an entire channel so that they no longer receive notifications from that specific category.

When you create a **`NotificationChannel`** *instance, you need to set an importance level, which determines how the system presents notifications from that channel to the user. The importance levels range from high (urgent and makes a sound) to low (no sound or visual interruption).*

The last step is to add our service to the **`AndroidManifest.xml`** file, inside the **`application`** tag:

```
<application
    android:allowBackup = "true"
    android:dataExtractionRules =
        "@xml/data_extraction_rules"
    android:fullBackupContent = "@xml/backup_rules"
    android:icon = "@mipmap/ic_launcher"
    android:label = "@string/app_name"
    android:supportsRtl = "true"
    android:theme = "@style/Theme.WhatsPackt"
    tools:targetApi = "31">
    <activity
        android:name = ".MainActivity"
        android:exported = "true"
        android:label = "@string/app_name"
        android:theme = "@style/Theme.WhatsPackt">
        <intent-filter>
            <action android:name=
                "android.intent.action.MAIN" />
            <category android:name =
                "android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
    <service
        android:name =
            "com.packt.data.WhatsPacktMessagingService"
        android:exported = "false">
        <intent-filter>
            <action android:name =
                "com.google.firebase.MESSAGING_EVENT" />
        </intent-filter>
    </service>
</application>
```

And with that, we have our app ready to receive push notifications.

In the next section, we are going to see how after all the work we have done to keep our code scalable and decoupled, we can easily use Firebase instead of the WebSocket to send and receive messages.

Replacing the Websocket with Firestore

As we saw in the previous section, Firebase is a powerful product that simplifies the implementation of the backend for our apps. Now, we are going to see how we can use it also to simplify the chat messages feature.

What is Firestore?

Firestore, more formally known as Cloud Firestore, is a flexible, scalable, and real-time NoSQL database provided by Firebase. Firestore is designed to store and sync data for client-side applications, making it an ideal choice for building modern, data-driven applications.

One of its most important features is the real-time data synchronization. Firestore automatically synchronizes data in real time across all connected clients, ensuring that your application's data is always up to date. This is especially useful for applications requiring real-time collaboration or live updates, such as our chat app.

It is important to note that as a NoSQL database, we would have first to define the data structure. How are we to structure our documents? Well, let's start with that.

Chat data structure

To handle chat messages in Firestore NoSQL, we can use the following structure:

- Create a collection called **`chats`**. Each document in this collection will represent a chat room or conversation between users. The document ID can be generated automatically by Firestore or created using a custom method (for example, a combination of user IDs). Here, we can include common data that we need for the conversation (think about our **`ChatRoom`** model), such as the user's name, avatars, and so on...
- For each chat document, create a subcollection called **`messages`**. This subcollection will store the individual messages for that chat room or conversation.
- Each document in the **`messages`** subcollection will represent a single message. The structure of a message document might include fields such as **`senderId`**, **`senderName`**, **`content`**, and **`timestamp`**.

Following that, our structure will look like this:

```
chats (collection)
  |
  └── chatId1 (document)
        |
        ├── users (subcollection)
        |   |
        |   ├── userId1 (document)
        |   |   ├── userId: "user1"
        |   |   ├── avatarUrl:
                      "https://example.com/avatar1.jpg"
        |   |   └── name: "John Doe"
        |   |
        |   └── userId2 (document)
        |       ├── userId: "user2"
        |       ├── avatarUrl:
                      "https://example.com/avatar2.jpg"
        |       └── name: "Jane Smith"
        |
        └── messages (subcollection)
              |
              ├── messageId1 (document)
```

```
                │   ├── senderId: "user1"
                │   ├── senderName: "John Doe"
                │   ├── content: "Hello, how are you?"
                │   └── timestamp: 1648749123
                │
                └── messageId2 (document)
                      ├── senderId: "user2"
                      ├── senderName: "Jane Smith"
                      ├── content: "I'm doing great! How
                                    about you?"
                      └── timestamp: 1648749156
```

One important aspect is that, ideally, we should have authentication set up to identify our users. We will learn how to build it in ***Chapter 7***, but for now, we are assuming that our users will be authenticated in Firebase.

Assuming that our chat will be used by authenticated users, we can limit and restrict access to the chat collection for modifications only for users who have already been authenticated. To accomplish that, we can define a set of rules in Firestore, using the Firebase console. Here is an example:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Allow authenticated users to create chat documents,
       but not modify or delete them
    match /chats/{chatId} {
      allow create: if request.auth != null;
      allow read, update, delete: if false;
    }
    // Allow chat participants to read the chat's user data
    match /chats/{chatId}/users/{userId} {
      allow read: if request.auth != null &&
        request.auth.uid in resource.data.userId;
      allow write: if false;
    }
    // Allow authenticated users to create/modify messages
```

```
        in a chat they are participating in
    match /chats/{chatId}/messages/{messageId} {
      // Get chat participants
      function isChatParticipant() {
        let chatUsersDoc = get(
            /databases/$(database)/documents/chats/
                $(chatId)/users/$(request.auth.uid));
        return chatUsersDoc.exists();
      }
      // Check if the sender is the authenticated user
      function isSender() {
        return request.auth != null && request.auth.uid ==
          request.resource.data.senderId;
      }
      allow create: if isChatParticipant() && isSender();
      allow read: if isChatParticipant();
      allow update, delete: if false;
    }
  }
}
```

Now that we have defined these rules, we can switch to our Android app code and create a **`FirestoreMessagesDataSource`** class.

Creating a FirestoreMessagesDataSource class

The first step to creating the **`FirestoreMessagesDataSource`** class is to create the model that we are going to use to serialize the documents. This model has to include the same fields we included when we designed the **`Message`** document structure:

```
import com.google.firebase.Timestamp
import com.google.firebase.firestore.PropertyName
import com.packt.feature.chat.domain.models.Message
import java.text.SimpleDateFormat
import java.util.*
data class FirestoreMessageModel(
```

```
    @Transient
    val id: String = "",
    @get:PropertyName("senderId")
    @set:PropertyName("senderId")
    var senderId: String = "",
    @get:PropertyName("senderName")
    @set:PropertyName("senderName")
    var senderName: String = "",
    @get:PropertyName("senderAvatar")
    @set:PropertyName("senderAvatar")
    var senderAvatar: String = "",
    @get:PropertyName("content")
    @set:PropertyName("content")
    var content: String = "",
    @get:PropertyName("timestamp")
    @set:PropertyName("timestamp")
    var timestamp: Timestamp = Timestamp.now()
)
```

Note that we are including a field called **`id`** that has the **`@Transient`** annotation – this field will store the document **`id`** value (that for us will be the unique identification for the message as every message has its own document). The reason we have to put the **`@Transient`** annotation is to avoid this **`id`** field being stored in the document itself when writing in Firestore.

Now, as we did with the **`MessagesSocketDataSource`** class, we need to convert this data model into the domain model. We already have the **`messages`** domain model, so, in this case, we only have to implement the function to convert the **`FirestoreMessageModel`** data class into our **`Message`** domain model:

```
fun toDomain(userId: String): Message {
    return Message(
        id = id,
        senderName = senderName,
        senderAvatar = senderAvatar,
        isMine = userId == senderId,
```

```
            contentType = Message.ContentType.TEXT,
            content = content,
            contentDescription = "",
            timestamp = timestamp.toDateString()
        )
    }
    private fun Timestamp.toDateString(): String {
        // Create a SimpleDateFormat instance with the desired
           format and the default Locale
        val formatter = SimpleDateFormat("dd/MM/yyyy HH:mm:ss",
            Locale.getDefault())
        // Convert the Timestamp to a Date object
        val date = toDate()
        // Format the Date object using the SimpleDateFormat
           instance
        return formatter.format(date)
    }
```

In this case, we are supposing we are only going to have text messages (no images) for simplification. However, it could have been easily done by including a field in the **`Firestore`** model indicating the type of message. Almost all the mapping between properties is straightforward, with the exception of the timestamp. In the **`Message`** model, we are expecting a **`String`** object with the date and time, and we are getting a **`Timestamp`** object from Firestore. So, we are using the **`Timestamp.toDateString()`** extension to obtain the formatted **`String`** object from the **`Timestamp`** object.

Also, as we would want to send messages too, we need to convert a domain **`Message`** object into the data object:

```
    companion object {
        fun fromDomain(message: Message): FirestoreMessageModel
        {
            return FirestoreMessageModel(
                id = "",
                senderName = message.senderName,
                senderAvatar = message.senderAvatar,
```

```
            content = message.content
        )
    }
}
```

Note that we are not setting the timestamp (it will be created when the object is created), and the **`id`** field doesn't have a real value (as it won't be stored in Firestore).

Now, we can proceed with the **`FirestoreMessagesDataSource`** implementation. First, we define the class and its dependency:

```
class FirestoreMessagesDataSource @Inject constructor(
    private val firestore: FirebaseFirestore =
        FirebaseFirestore.getInstance()
) {
```

Then, we are going to add a **`getMessages`** function, to obtain chat messages:

```
    fun getMessages(chatId: String, userId: String):
    Flow<Message> = callbackFlow {
```

Inside this function, we will get a reference to the **`messages`** subcollection inside the specified chat:

```
        val chatRef =
            firestore.collection("chats").document(chatId)
                .collection("messages")
```

Now, we will create a query to get the messages ordered by timestamp (ascending):

```
        val query = chatRef.orderBy("timestamp",
            Query.Direction.ASCENDING)
```

In the next step, we add a snapshot listener to the query to listen for real-time updates. Every time a document in the messages is added, we will get a snapshot of the changed document there so that we can emit it through the flow to the consumers connected (in our case, **`MessagesRepository`**):

```
val listenerRegistration =
query.addSnapshotListener { snapshot, exception ->
    // If there's an exception, close the Flow with
       the exception
    if (exception != null) {
        close(exception)
        return@addSnapshotListener
    }
```

Just before sending the new messages through the flow, we need to map them to their domain counterpart and provide their ID. Also, **`userId`** will be needed to identify if the user has written the new message or if it is written by the other user in the conversation:

```
    val messages = snapshot?.documents?.mapNotNull
    { doc ->
        val message =
            doc.toObject(FirestoreMessageModel::
            class.java)
        message?.copy(id = doc.id) // Copy the
                                      message with
                                      the document
                                      ID
    } ?: emptyList()
    val domainMessages = messages.map {
        it.toDomain(userId) }
```

Finally, we can send the list of messages to **`Flow`**:

```
    domainMessages.forEach {
```

```
                try {
                    trySend(it).isSuccess
                } catch (e: Exception) {
                    close(e)
                }
            }
        }
```

In the case **`Flow`** is no longer needed, we should remove the snapshot listener:

```
        awaitClose { listenerRegistration.remove() }
    }
```

We also need to add a function to send messages. To send a message, we will simply add it to the **`messages`** collection in the document with the **`chatId`** value of the related conversation:

```
    fun sendMessage(chatId: String, message: Message) {
        val chatRef =
            firestore.collection("chats").document(chatId)
                .collection("messages")
        chatRef.add(FirestoreMessageModel
            .fromDomain(message))
    }
}
```

Next, we need to replace our previous **`MessagesSocketDataSource`** instance in **`MessagesRepository`** with **`FirestoreMessagesDataSource`**:

```
class MessagesRepository @Inject constructor(
    //private val dataSource: MessagesSocketDataSource
    private val dataSource: FirestoreMessagesDataSource
): IMessagesRepository {
    override suspend fun getMessages(chatId: String,
    userId: String): Flow<Message> {
        return dataSource.getMessages(chatId, userId)
```

```
    }
    override suspend fun sendMessage(chatId: String,
    message: Message) {
        dataSource.sendMessage(chatId, message)
    }
    override suspend fun disconnect() {
        // do nothing, Firestore data source is
           disconnected as soon as the flow has no
           subscribers
    }
}
```

And with some minor changes, we will have integrated this new provider. The good thing is that, as we have been working following a Clean Architecture, with mappings between layers, we don't have to change anything in other layers; for example, in **`Usecases`**, **`ViewModel`**, or the UI (apart from providing the **`chatId`** value and the **`userId`** value when calling the **`getMessages`** and **`sendMessage`** methods).

We could also have the two data sources living together in the same app (one as a fallback of the other), as the role of the repository is to serve as an orchestrator of the different data sources for a certain entity (in this case, the messages). We will see more about this in the next chapter as we will want to add local storage to our messages.

Summary

In this chapter, we explored various aspects of building a messaging app for Android. We discussed different approaches for sending and receiving messages, such as using WebSockets with Ktor or Firebase Firestore. We also covered how to structure the app using Clean Architecture principles, with separate layers for data, domain, and presentation, to ensure a well-organized and maintainable code base, and saw how easy is to introduce changes (for example, a change in the messages provider) if our architecture components are well decoupled.

Then, we delved into handling connection errors and synchronization issues using Kotlin coroutines and Flow, implementing error handling and retry mechanisms for a seamless user experience. Additionally, we examined the importance of push notifications in messaging apps and demonstrated their implementation using FCM, from setting up FCM in a project to handling incoming notifications.

By the end of this chapter, you should have a comprehensive understanding of the components and techniques required to build a robust real-time messaging app on Android.

Now, let's move on to learn how we can optimize our WhatsPackt app so that we can back up messages.